

CSE 331

AUTOMATA AND COMPUTABILITY

Lecture 4

Spring 2026

Today's learning goals

Sipser Ch 1.1, 1.2

- Distinguish between an NFA and a DFA
- Define nondeterminism and why it may be helpful
- Design NFA recognizing a given language
- Precisely specify general constructions for NFA recognizing languages output by operations

General proof structure/strategy

Theorem: For any L over Σ , if L is regular then [the result of some operation on L] is also regular.

Proof:

Given name variables for sets, machines assumed to exist.

WTS state goal and outline plan.

Construction using objects previously defined + new tools working towards goal. Give formal definition and explain.

Correctness prove that construction works.

Conclusion recap what you've proved.

Regular languages

Over any fixed alphabet,

- Each finite language is regular
- There are infinite regular languages
- If a set is regular, its complement is also regular
- If two sets are regular, their union is also regular
- If two sets are regular, their intersection is also regular

The regular operations

Sipser Def 1.23 p. 44

For A, B languages over same alphabet, define:

$$A \cup B = \{x \mid x \in A \text{ or } x \in B\} \quad \checkmark$$

$$A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$$

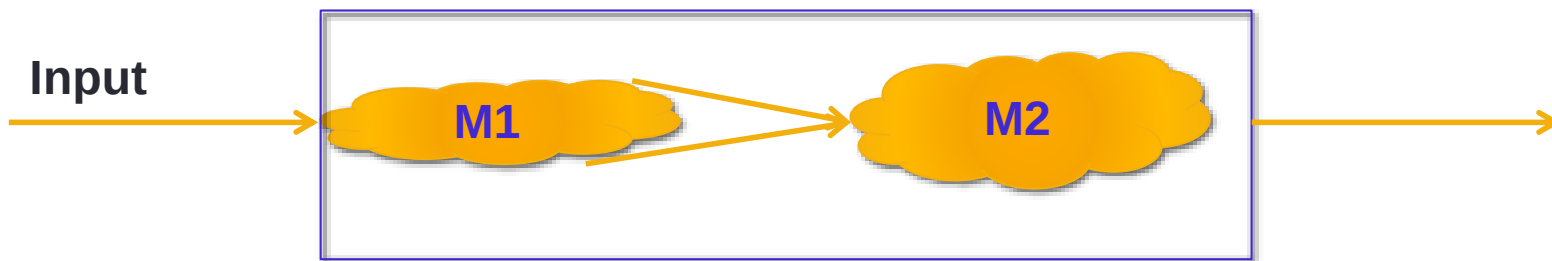
$$A^* = \{x_1x_2 \dots x_k \mid k \geq 0 \text{ and each } x_i \in A\}$$



How can we prove that the concatenation of two regular languages is a regular language?

Construction

$$A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$$



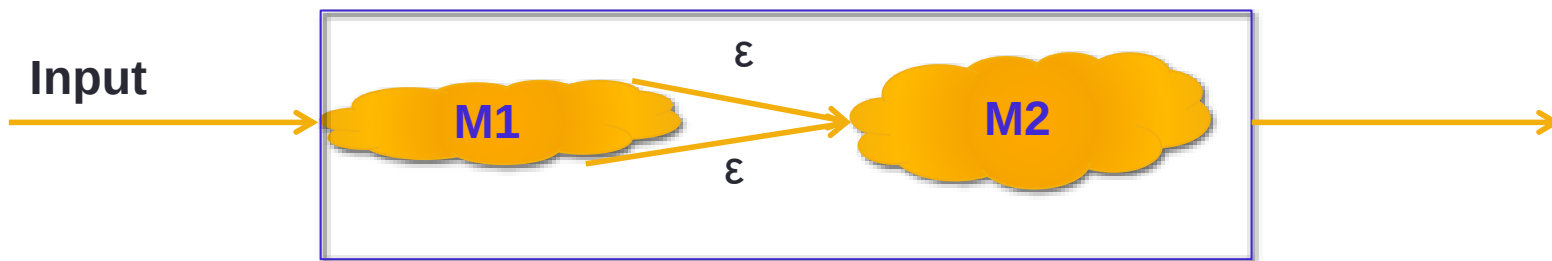
How would this work for $A = \{a^n \mid n \geq 0\}$, $B = \{b^m \mid m \geq 0\}$?

Solution: nondeterminism

- Allow transitions that don't "consume" any input
- Instead of exactly one outgoing transition from each state labelled by each symbol
 - Can have 0 transitions
 - Can have > 1 transitions

Construction

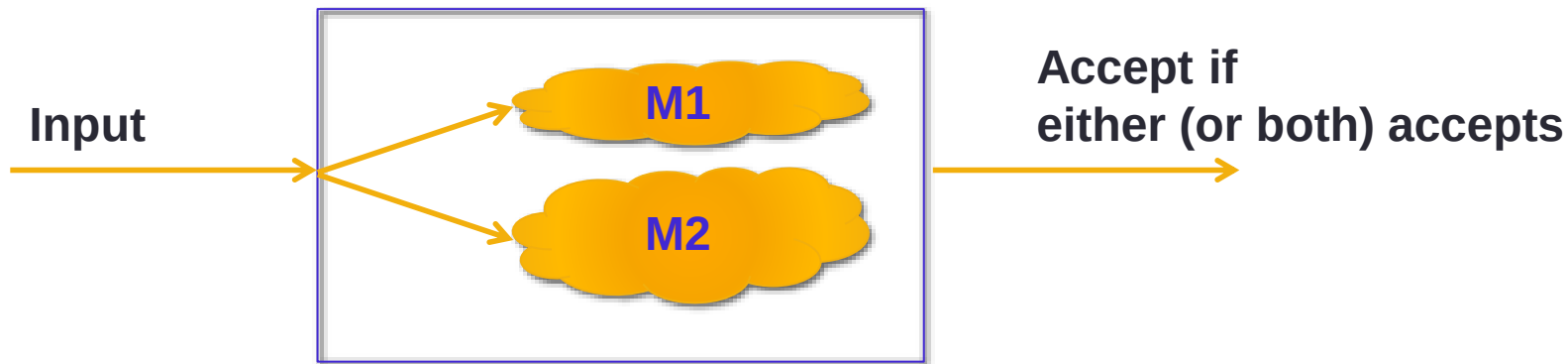
$$A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$$



How would this work for $A = \{ a^n \mid n \geq 0 \}$, $B = \{ b^m \mid m \geq 0 \}$?

New – easier – construction for union

- "Guess" one of finite list of criteria to meet



Caveat

- This nondeterminism trick doesn't help us with closure arguments.

Why?

- To prove that the **class of regular languages** is closed under OP, needs that $OP(L)$ is recognized by a **DFA**

Caveat

- This nondeterminism trick doesn't help us with closure arguments **YET!**

Why?

- We'll prove that any language recognized by some NFA is also recognized by some (different) DFA.

Differences between NFA and DFA

- **DFA**: unique computation path for each input
- **NFA**: allow several (or zero) alternative computations on *same input*
 - $\delta(q,x)$ may specify **more than one** possible next states
 - ϵ transitions allow the machine to **transition between states spontaneously**, without consuming any input symbols

Formal definition of NFA

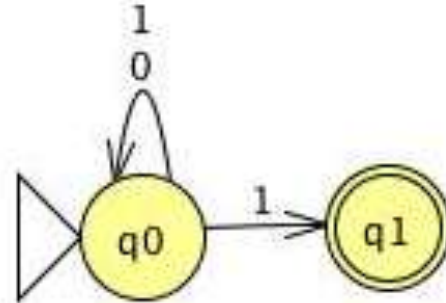
A **nondeterministic finite automaton** is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where

1. Q is a finite set called the states
2. Σ is a finite set called the alphabet
3. $\delta : Q \times \Sigma_{\epsilon} \rightarrow \mathcal{P}(Q)$ is the transition function
4. $q_0 \in Q$ is the start state
5. $F \subseteq Q$ is the set of accept states.

Which piece of the definition of NFA means there might be **more than one** possible next state from a given state, when reading symbol x from the alphabet?

- A. Line 2, the size of Σ
- B. Line 3, the domain of δ
- C. Line 3, the codomain of δ
- D. Line 5, that F is a set
- E. I don't know.

Tracing NFA execution



- Is 0 accepted?
- Is 1 accepted?
- Is 0101 accepted?
- Is 110 accepted?
- Is the empty string accepted?

Acceptance in an NFA

An NFA $(Q, \Sigma, \delta, q_0, F)$ accepts a string w in Σ^* iff we can write $w = y_1 y_2 \cdots y_m$ where each $y_i \in \Sigma_\epsilon$ and **there is** a sequence of states $r_0, \dots, r_m \in Q$ such that

1. $r_0 = q_0$
2. $r_{i+1} \in \delta(r_i, y_{i+1})$ for each $i = 0, \dots, m - 1$
3. $r_m \in F$.

More differences between NFA and DFA

- **DFA**: unique computation path for each input
- **NFA**: allow several (or zero) alternative computations on *same input*
 - $\delta(q,x)$ may specify **more than one** possible next states
 - ϵ transitions allow the machine to **transition between states spontaneously**, without consuming any input symbols
- Types of components of formal definition
 - **DFA** $\delta : Q \times \Sigma \rightarrow Q$
 - **NFA** $\delta : Q \times \Sigma_{\epsilon} \rightarrow \mathcal{P}(Q)$

Similarities between DFA and NFA

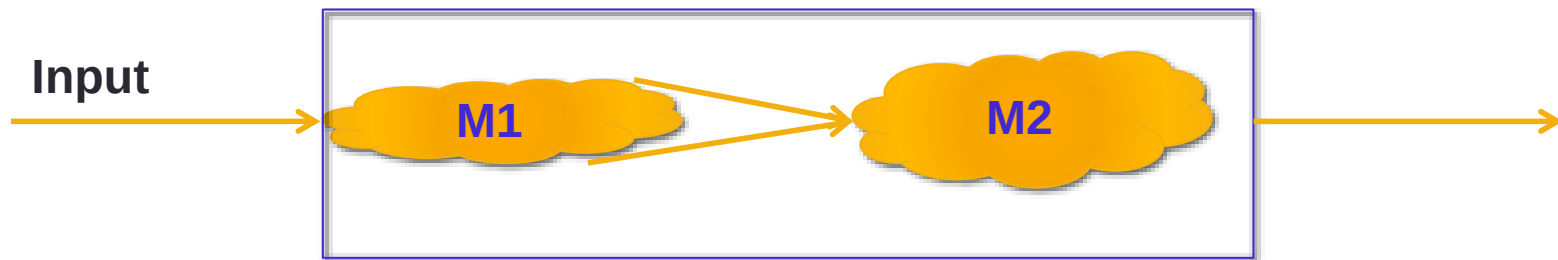
- If L is a language recognized by a DFA, is there some NFA that recognizes it?
 - A. Yes
 - B. No
 - C. Depends on L
 - D. I don't know.

Next steps

- Defining NFA in closure constructions formally.
- Showing that NFA and DFA are equally expressive

Concatenation

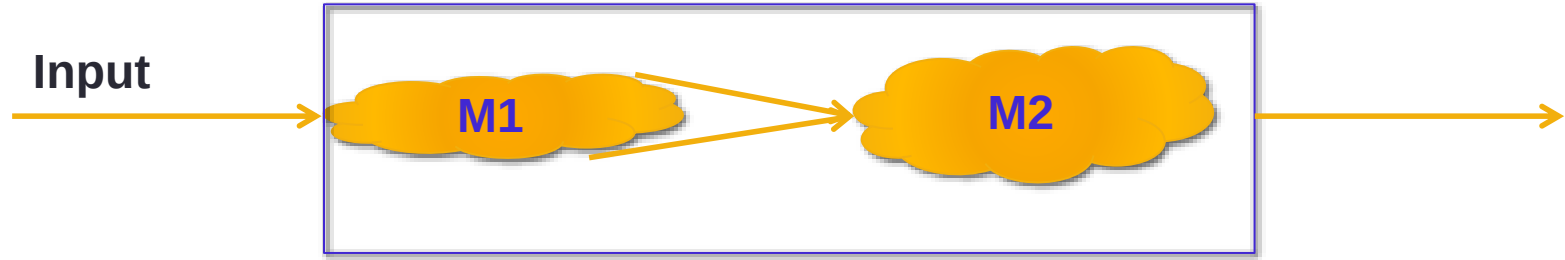
- "Guess" some stage of input at which switch modes



Given DFAs $M1 = (Q1, \Sigma, \delta1, q1, F1)$, $M2 = (Q2, \Sigma, \delta2, q2, F2)$

build NFA $N = (Q1 \cup Q2, \Sigma, \delta, q1, F2)$ with $\delta \dots$

Concatenation



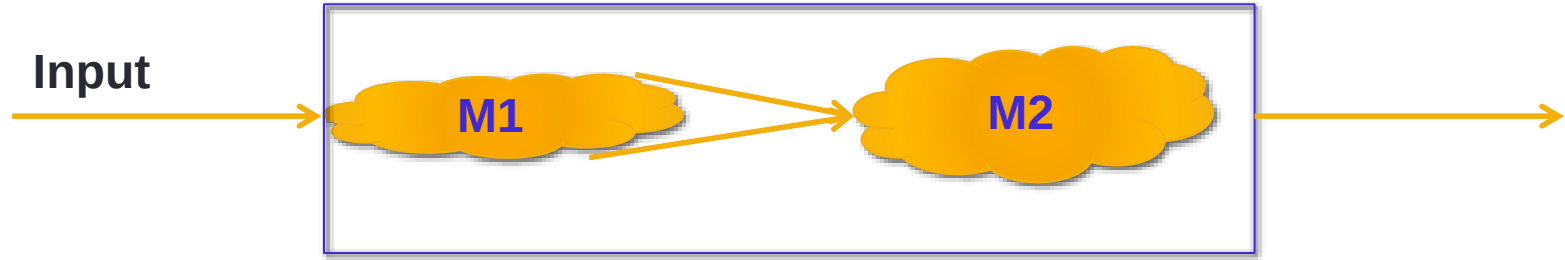
$\delta(q, x) =$

if q is in Q_1 , x is in Σ

if q is in Q_2 , x is in Σ

?

Concatenation

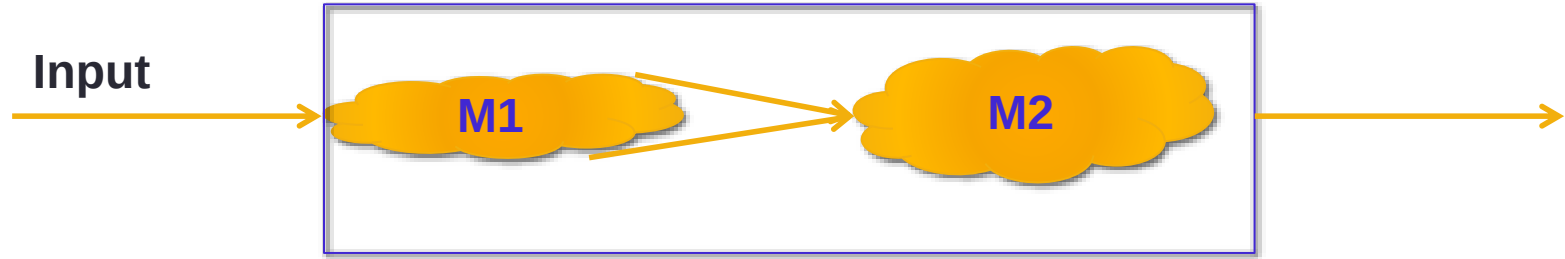


$$\delta(q, x) = \{\delta_1(q, x)\}$$
$$\{\delta_2(q, x)\}$$

if q is in Q_1 , x is in Σ

if q is in Q_2 , x is in Σ

Concatenation

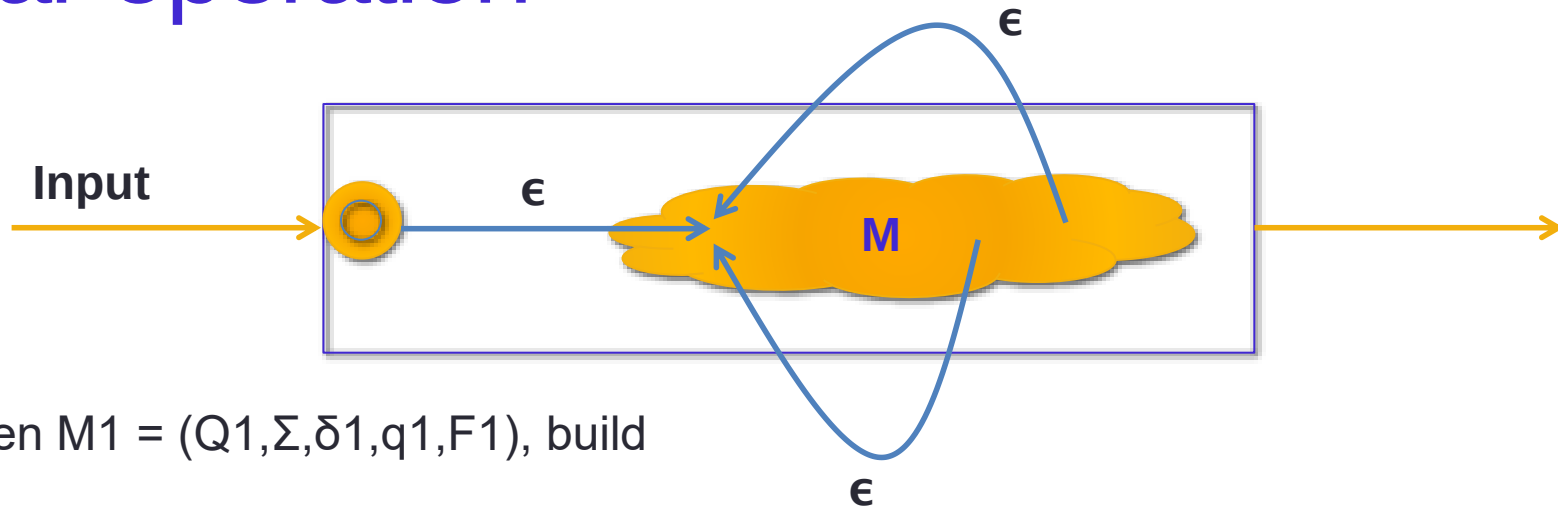


$$\delta(q, x) = \begin{cases} \delta_1(q, x) \\ \delta_2(q, x) \\ q_2 \\ \emptyset \end{cases}$$

if q is in Q_1 , x is in Σ
if q is in Q_2 , x is in Σ
if q is in F_1 , $x = \epsilon$
otherwise

Correctness proof in the book (page 61)

Star operation



Given $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$, build

$$N = (Q_1 \cup \{q_0\}, \Sigma, \delta, q_0, F_1 \cup \{q_0\})$$

and $\delta(q, x) = \dots$

Construction in the book (page 63)

Next steps

- Showing that NFA and DFA are equally expressive.